

KB001 - Payment Gateway Timeout Troubleshooting Runbook

Document Metadata

Field	Value
Document ID	KB001
Document Type	Production Support Runbook / Known Error Document
Service Name	Payment Gateway Service
Application Owner	Payments Engineering Team
Support Group	Payments SRE L2
Severity Coverage	P1, P2
Environment	Production, Staging
Last Updated	2026-01-15
Review Cycle	Quarterly

1. Overview

The Payment Gateway Service is responsible for processing customer payment transactions by coordinating communication between customer applications, internal order management systems, fraud detection services, banking partners, and third-party payment processors.

Payment timeout incidents occur when the Payment Gateway fails to complete transaction processing within the expected service-level timeout window.

These incidents typically impact:

- Customer checkout experience
- Payment authorization flow
- Transaction settlement
- Refund processing
- Merchant payment APIs

Common business impact:

- Customers unable to complete purchases
- Increase in failed transaction rate
- Revenue loss during outage period

- Increased customer support tickets
-

2. Common Incident Symptoms

Payment Gateway timeout incidents commonly appear with the following symptoms.

Application Symptoms

- Payment transactions remain stuck in "PROCESSING" state
- Customers receive timeout errors during checkout
- Payment confirmation is delayed
- Duplicate payment attempts increase
- Refund processing becomes slow

API Symptoms

Common API errors:

```
HTTP 504 Gateway Timeout
```

```
HTTP 503 Service Unavailable
```

```
HTTP 500 Internal Server Error
```

Example application logs:

```
ERROR PaymentProcessor
```

```
Transaction failed.
```

```
Exception:  
java.net.SocketTimeoutException
```

```
Message:  
Read timed out while waiting for payment provider response
```

Another common error:

```
ConnectionTimeoutException:
```

```
Unable to acquire database connection within configured timeout.
```

3. Monitoring Indicators

Before starting RCA, verify the following monitoring metrics.

Payment Metrics

Metric	Warning Threshold	Critical Threshold
transaction_failure_rate	>5%	>15%
payment_latency_p95	>3 sec	>8 sec
payment_latency_p99	>5 sec	>15 sec
pending_transactions	>500	>2000

Infrastructure Metrics

Monitor:

- CPU utilization
 - Memory consumption
 - Database connections
 - Network latency
 - API response time
 - Queue size
-

4. Common Root Causes and Resolution Steps

RCA Pattern 1: Database Connection Pool Exhaustion

Description

The Payment Gateway maintains a fixed number of database connections.

During high traffic or connection leaks, all available connections may become occupied.

New payment requests wait for connections and eventually timeout.

Symptoms

Indicators:

- Sudden increase in API latency
- Database connection pool reaches maximum size
- Multiple payment APIs timeout together

Application logs:

```
HikariPool-1 - Connection is not available
request timed out after 30000ms
```

Database metrics:

```
active_connections = max_connections  
connection_wait_time > threshold
```

Validation Steps

Step 1:

Check active database connections.

```
SELECT  
count(*)  
FROM active_connections;
```

Step 2:

Identify long-running queries.

```
SELECT query, execution_time  
FROM database_sessions  
ORDER BY execution_time DESC;
```

Step 3:

Review application connection pool metrics.

Expected:

```
Active Connections < Maximum Connections
```

Immediate Resolution

1. Restart affected Payment Gateway instances.
2. Clear stale database connections.
3. Increase temporary connection pool limit.

Example:

```
payment-service:  
  
database:  
  maxPoolSize: 200  
  connectionTimeout: 30000
```

Permanent Fix

- Optimize slow database queries
 - Fix connection leaks
 - Implement connection monitoring alerts
-

RCA Pattern 2: Third Party Payment Provider Latency

Description

Payment Gateway depends on external banking/payment providers.

If a provider responds slowly, transactions remain pending until timeout.

Affected integrations:

- Bank authorization API
- Card payment network
- Wallet providers

Symptoms

Only specific payment methods failing.

Examples:

- Credit cards failing
- UPI working normally

or

- One banking provider impacted

Logs:

```
ExternalProviderTimeoutException
```

```
Provider response exceeded timeout limit
```

Validation

Check provider latency dashboard.

Metrics:

```
provider_response_time
```

```
provider_error_percentage
```

```
provider_timeout_count
```

Resolution

Short term:

- Enable backup payment provider
- Increase timeout threshold temporarily
- Disable unhealthy provider route

Long term:

- Implement circuit breaker
 - Add automatic failover
 - Improve provider monitoring
-

RCA Pattern 3: Recent Deployment Issue

Description

Payment failures frequently happen after new application releases.

Common reasons:

- Incorrect configuration
- API contract mismatch
- Database schema change
- Missing environment variables

Detection

Compare incident timestamp with deployment timestamp.

Check:

```
Deployment happened within last 24 hours?
```

Review deployment logs:

```
payment-service:v4.8.1 deployed  
configuration updated
```

Validation Steps

Check new error patterns:

```
grep ERROR payment-service.log
```

Compare:

Before deployment:

```
error_rate = 0.5%
```

After deployment:

```
error_rate = 20%
```

Resolution

If deployment caused impact:

1. Stop rollout
 2. Rollback application
 3. Restore previous configuration
 4. Verify transaction recovery
-

RCA Pattern 4: High Traffic / Capacity Issue

Description

Unexpected traffic spikes can overload Payment Gateway workers.

Common causes:

- Sale events
- Marketing campaigns
- Bot traffic
- Seasonal load increase

Symptoms:

```
CPU usage > 90%
```

```
request queue increasing
```

```
thread pool exhausted
```

Resolution

Immediate:

- Increase application replicas
- Enable autoscaling
- Increase worker threads

Example:

```
scale payment-service replicas=10
```

Permanent:

- Capacity planning
- Load testing
- Traffic forecasting

5. Incident Investigation Checklist

When a Payment Gateway timeout incident occurs:

Step 1

Identify impact.

Questions:

- How many users affected?
- Which payment methods failing?
- Is issue regional/global?

Step 2

Check recent changes.

Verify:

- Deployments
- Configuration updates
- Database changes

Step 3

Analyze dependencies.

Check:

- Database
- Payment providers
- Authentication service
- Network

Step 4

Apply mitigation.

Possible actions:

- Restart service

- Rollback release
 - Increase capacity
 - Enable fallback
-

6. Escalation Matrix

Condition	Escalate To
Database outage	Database Engineering
External provider failure	Payment Integration Team
Deployment issue	Application Development Team
Infrastructure issue	Cloud Operations Team

7. Related Historical Incidents

INC-10234

Issue:

Payment Gateway timeout during peak traffic.

Root Cause:

Database connection pool exhaustion.

Resolution:

Increased DB connection pool and optimized queries.

INC-10892

Issue:

Card payments failing intermittently.

Root Cause:

External payment provider latency.

Resolution:

Enabled secondary provider routing.

INC-11203

Issue:

Timeout errors after deployment.

Root Cause:

Incorrect database configuration.

Resolution:

Rolled back deployment and fixed configuration.

8. RCA Output Template

When generating RCA:

Include:

Root Cause

The primary technical reason behind failure.

Evidence

Mention:

- Similar incidents
- Logs
- Monitoring metrics
- KB references

Resolution Applied

Actions taken to restore service.

Preventive Actions

Long term fixes to prevent recurrence.

Example:

Root Cause:

Payment Gateway experienced database connection exhaustion.

Evidence:

- Active DB connections reached maximum limit.
- Similar historical incident INC-10234 identified.
- Application logs reported connection timeout.

Resolution:

Restarted payment instances and increased pool size.

Preventive Action:

Added connection pool monitoring alerts.

End of Document